

Internet de las Cosas

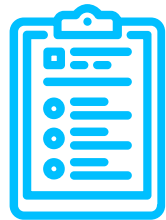
LAB 1 – Arduino

Instrucciones Digitales

*Ejemplos de Programación en **Arduino***

Sesión de Laboratorio

1.



Programación en Arduino

Instrucciones Digitales

Introducción a código C para Arduino

En la página de arduino, podemos encontrar diferentes referencias de códigos y programas pre-hechos para el desarrollo de diferentes soluciones. (<https://docs.arduino.cc/programming/>)

Programming

Learn all you need to know about the Arduino programming language as well as other compatible languages.

🔗 Language Reference

Serial
Wire
Print
attachInterrupt()
pinMode()
String()
view all →

📖 Libraries

LiquidCrystal
Servo
LiquidCrystal I2C
WiFi
SD
Stepper
view all →

📁 Built-in Examples

How to Wire and Program a Button
Blink
If Statement (Conditional Statement)
Arduino as ISP and Arduino Bootloaders
Analog Read Serial
String to Int Function
view all →



Introducción a código C para Arduino

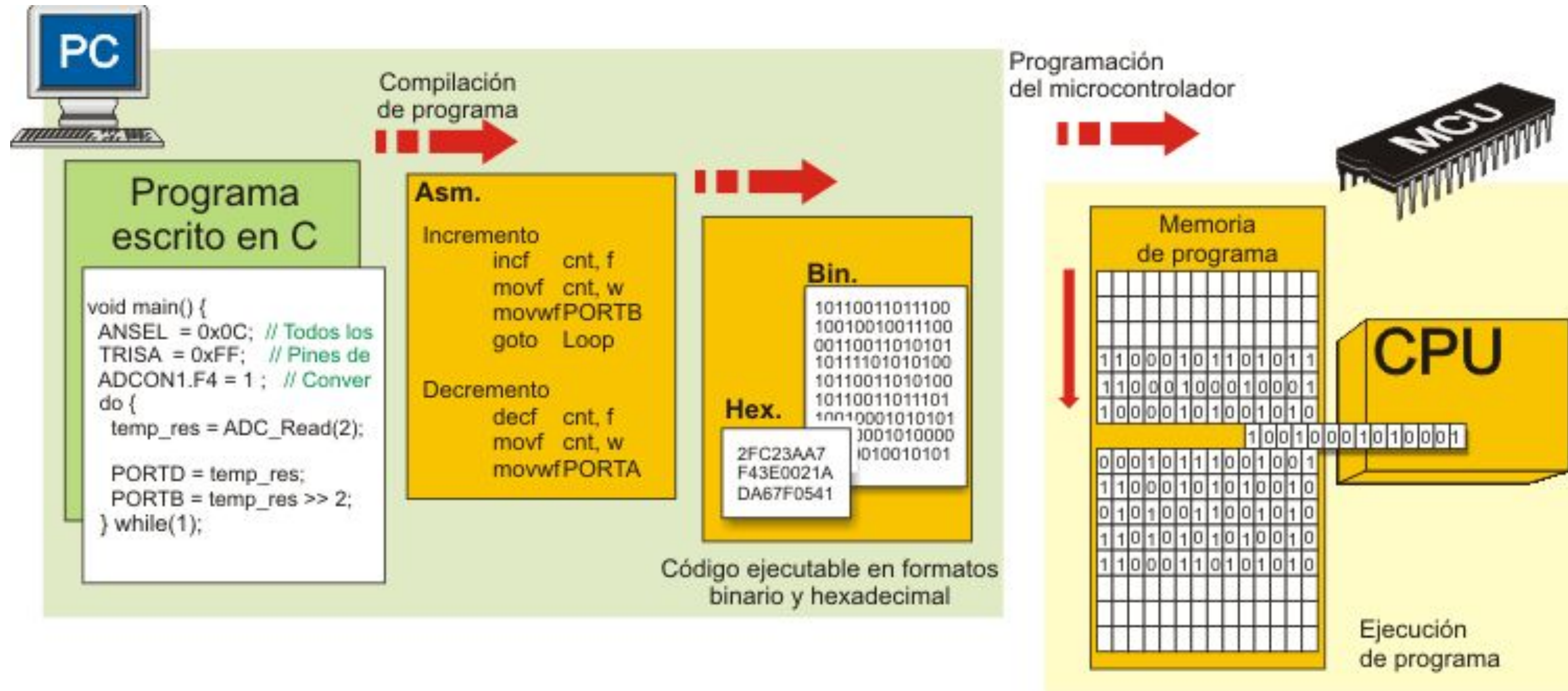
Lenguaje para Arduino

El lenguaje de programación de Arduino es C++. No es un C++ puro sino que es una adaptación que proviene de avr-libc que provee de una librería de C de alta calidad para usar con GCC (compilador de C y C++) en los microcontroladores AVR de Atmel y muchas utilidades específicas para las MCU AVR de Atmel como avrdude.

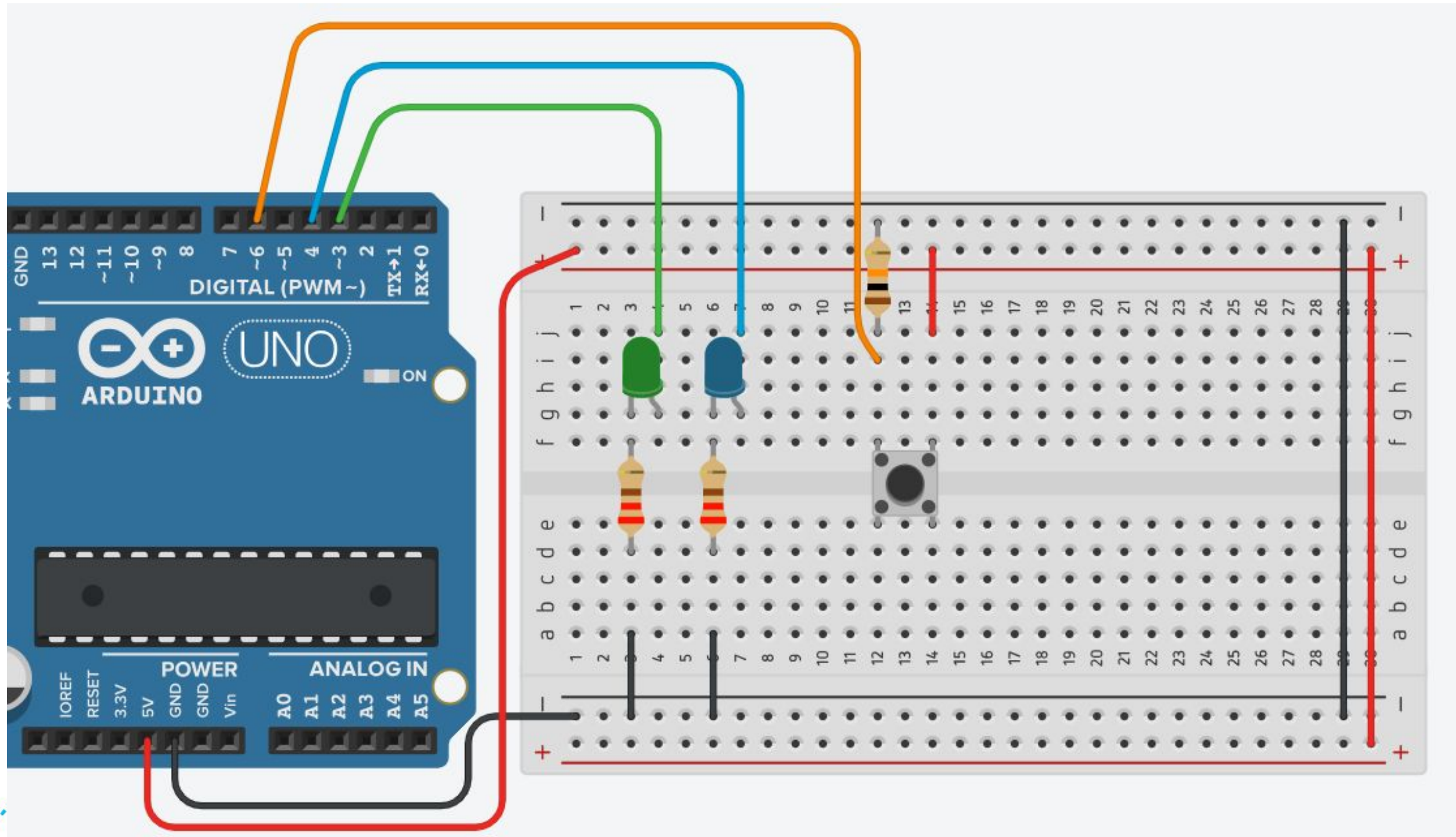
Aunque se hable de que hay un lenguaje propio de programación de Arduino, no es cierto, la programación se hace en C++ pero Arduino ofrece una api o core que facilitan la programación de los pines de entrada y salida y de los puertos de comunicación, así como otras librerías para operaciones específicas.



Lenguaje para Arduino



EJEMPLO I. Entradas y Salidas Digitales



a. Configuración Digital (GPIO)

GPIO significa entradas y salidas de propósito general (en inglés General Purpose Input Output). Estos pines se pueden definir dentro del código para dos tipos de función: lectura y escritura.

Es necesario conocer la siguiente sintaxis.

- **pinMode()**

Configura el pin especificado para que se comporte como entrada o como salida.

Syntax

```
pinMode(pin, mode)
```

Parameters

`pin`: the Arduino pin number to set the mode of.

`mode`: INPUT, OUTPUT, or INPUT_PULLUP. See the [Digital Pins](#) page for a more complete description of the functionality.



a. Configuración Digital (GPIO)

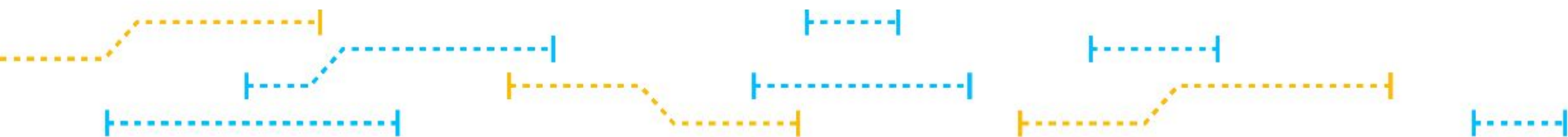
- **pinMode()**

Configura el pin especificado para que se comporte como entrada o como salida.

Example Code

The code makes the digital pin 13 OUTPUT and Toggles it HIGH and LOW

```
void setup() {  
  pinMode(13, OUTPUT);    // sets the digital pin 13 as output  
}  
  
void loop() {  
  digitalWrite(13, HIGH); // sets the digital pin 13 on  
  delay(1000);            // waits for a second  
  digitalWrite(13, LOW);  // sets the digital pin 13 off  
  delay(1000);            // waits for a second  
}
```



a. Configuración Digital (GPIO)

- **digitalWrite()**

Escribe un valor **ALTO** o **BAJO** en un pin digital.

Si el pin se ha configurado como **OUTPUT** con `pinMode()`, su voltaje se ajustará al valor correspondiente: **5V (o 3,3V en placas de 3,3V) para HIGH, 0V (Tierra) para LOW.**

Si el pin está configurado como **INPUT**, `digitalWrite()` activará (HIGH) o desactivará (LOW) el pullup interno del pin de entrada. Se recomienda ajustar `pinMode()` a `INPUT_PULLUP` para habilitar la resistencia interna de pull-up. *Vea el tutorial Pines Digitales para más información.*



a. Configuración Digital (GPIO)

- **digitalWrite()**

Escribe un valor **ALTO** o **BAJO** en un pin digital.

Example Code

Syntax

```
digitalWrite(pin, value)
```

Parameters

pin: the Arduino pin number.

value: HIGH OR LOW.

The code makes the digital pin 13 an OUTPUT and toggles it by alternating between HIGH and LOW at one second pace.

```
void setup() {  
  pinMode(13, OUTPUT);    // sets the digital pin 13 as output  
}  
  
void loop() {  
  digitalWrite(13, HIGH); // sets the digital pin 13 on  
  delay(1000);            // waits for a second  
  digitalWrite(13, LOW);  // sets the digital pin 13 off  
  delay(1000);            // waits for a second  
}
```



5. Configuración y modos de uso de pines

a. Configuración Digital (GPIO)

- **digitalRead()**

Lee el valor de un pin digital especificado, ya sea **HIGH** o **LOW**.

Syntax

```
digitalRead(pin)
```

Parameters

pin: the Arduino pin number you want to read

Example Code

Sets pin 13 to the same value as pin 7, declared as an input.

```
int ledPin = 13; // LED connected to digital pin 13
int inPin = 7;   // pushbutton connected to digital pin 7
int val = 0;     // variable to store the read value

void setup() {
  pinMode(ledPin, OUTPUT); // sets the digital pin 13 as output
  pinMode(inPin, INPUT);   // sets the digital pin 7 as input
}

void loop() {
  val = digitalRead(inPin); // read the input pin
  digitalWrite(ledPin, val); // sets the LED to the button's value
}
```

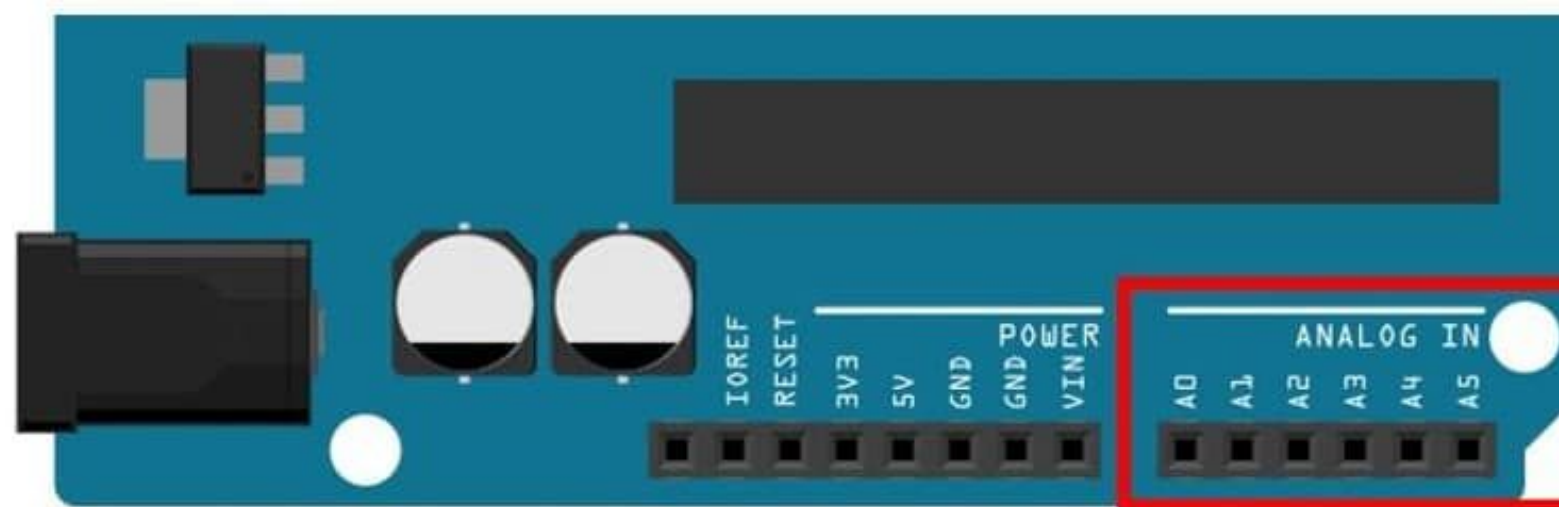
5. Configuración y modos de uso de pines

b. Configuración Analógica

Permite realizar una lectura de un valor analógico de voltaje en los pines del tipo Analog. El valor de voltaje medido no debe superar los 3.3V o 5V dependiendo de la placa. Emplea un ADC interno en la placa, cuya resolución depende del modelo.

- **analogRead()**

Lee el valor del pin analógico especificado. Las placas Arduino contienen un convertidor analógico-digital multicanal de 10 bits. Esto significa que mapeará los voltajes de entrada entre 0 y el voltaje de funcionamiento (5V o 3.3V) en valores enteros entre 0 y 1023.



Analog pins

NSFORMATEC

b. Configuración Analógica

Permite realizar una lectura de un valor analógico de voltaje en los pines del tipo Analog. El valor de voltaje medido no debe superar los 3.3V o 5V dependiendo de la placa. Emplea un ADC interno en la placa, cuya resolución depende del modelo.

- **analogRead()**

Lee el valor del pin analógico especificado. Las placas Arduino contienen un convertidor analógico-digital multicanal de 10 bits. Esto significa que mapeará los voltajes de entrada entre 0 y el voltaje de funcionamiento (5V o 3.3V) en valores enteros entre 0 y 1023.



b. Configuración Analógica

- **analogRead()**

BOARD	OPERATING VOLTAGE	USABLE PINS	MAX RESOLUTION
UNO R3	5 Volts	A0 to A5	10 bits
UNO R4 (Minima, WiFi)	5 Volts	A0 to A5	14 bits**
Mini	5 Volts	A0 to A7	10 bits
Nano, Nano Every	5 Volts	A0 to A7	10 bits
Nano 33 (IoT, BLE, RP2040, ESP32)	3.3 Volts	A0 to A7	12 bits**
Mega, Mega2560, MegaADK	5 Volts	A0 to A14	10 bits
Micro	5 Volts	A0 to A11*	10 bits
Leonardo	5 Volts	A0 to A11*	10 bits
Zero	3.3 Volts	A0 to A5	12 bits**
Due	3.3 Volts	A0 to A11	12 bits**
GIGA R1	3.3 Volts	A0 to A11	16 bits**
MKR Family boards	3.3 Volts	A0 to A6	12 bits**

b. Configuración Analógica

- **analogRead()**

Syntax

```
analogRead(pin)
```

Parameters

pin: the name of the analog input pin to read from.

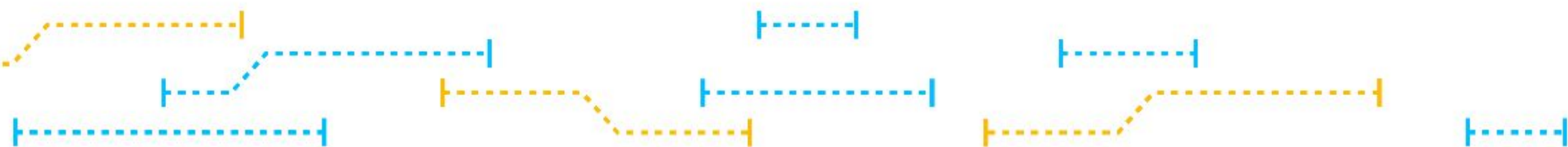
Example Code

The code reads the voltage on analogPin and displays it.

```
int analogPin = A3; // potentiometer wiper (middle terminal) connected to analog pin 3
                    // outside leads to ground and +5V
int val = 0; // variable to store the value read

void setup() {
  Serial.begin(9600);          // setup serial
}

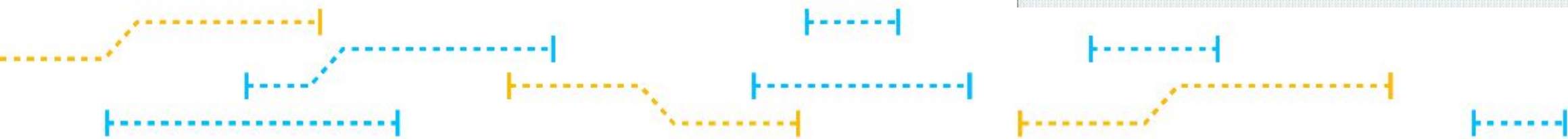
void loop() {
  val = analogRead(analogPin); // read the input pin
  Serial.println(val);         // debug value
}
```



5. Configuración y modos de uso de pines

b. Configuración Analógica

- `analogRead()`



c. Comunicación Serial

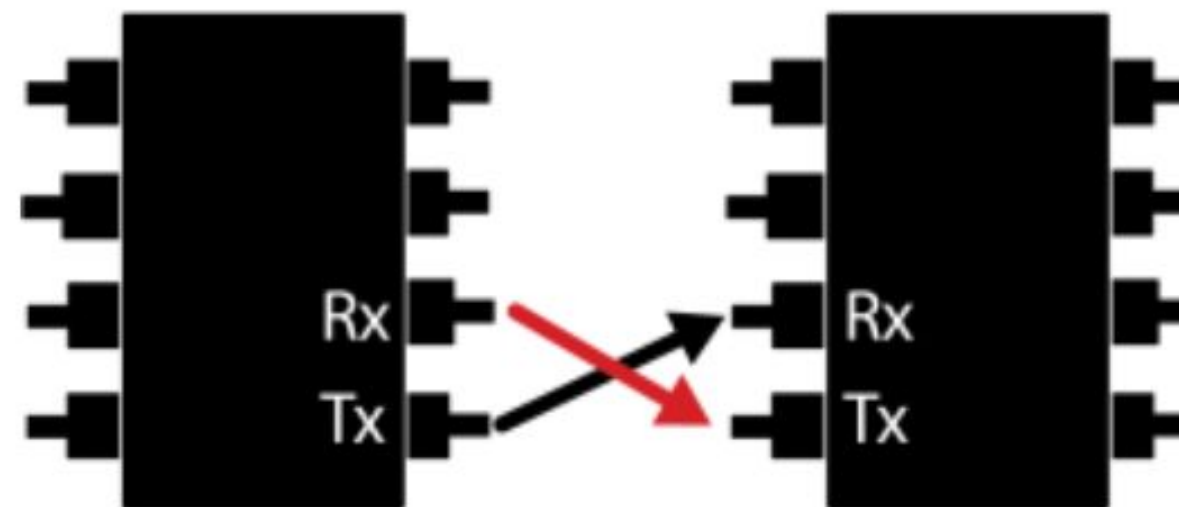
Se utiliza para la comunicación entre la placa Arduino y un ordenador u otros dispositivos **(como algunos sensores)**. Todas las placas Arduino tienen al menos un puerto serie (también conocido como UART o USART), y algunas tienen varios. Puede utilizar el monitor serie incorporado en el entorno Arduino para comunicarse con una placa Arduino.

La comunicación serie en los pines TX/RX utiliza niveles lógicos **TTL (5V o 3.3V dependiendo de la placa)**. No conecte estos pines directamente a un puerto serie RS232; funcionan a +/- 12V y pueden dañar su placa Arduino.



c. Comunicación Serial

Tarjeta Arduino		Serial		Serial1		Serial2		Serial3	
		RX	TX	RX	TX	RX	TX	RX	TX
UNO R3, Mini, Nano		0	1	X	X	X	X	X	X
MEGA R3		0	1	19	18	16	17	15	14

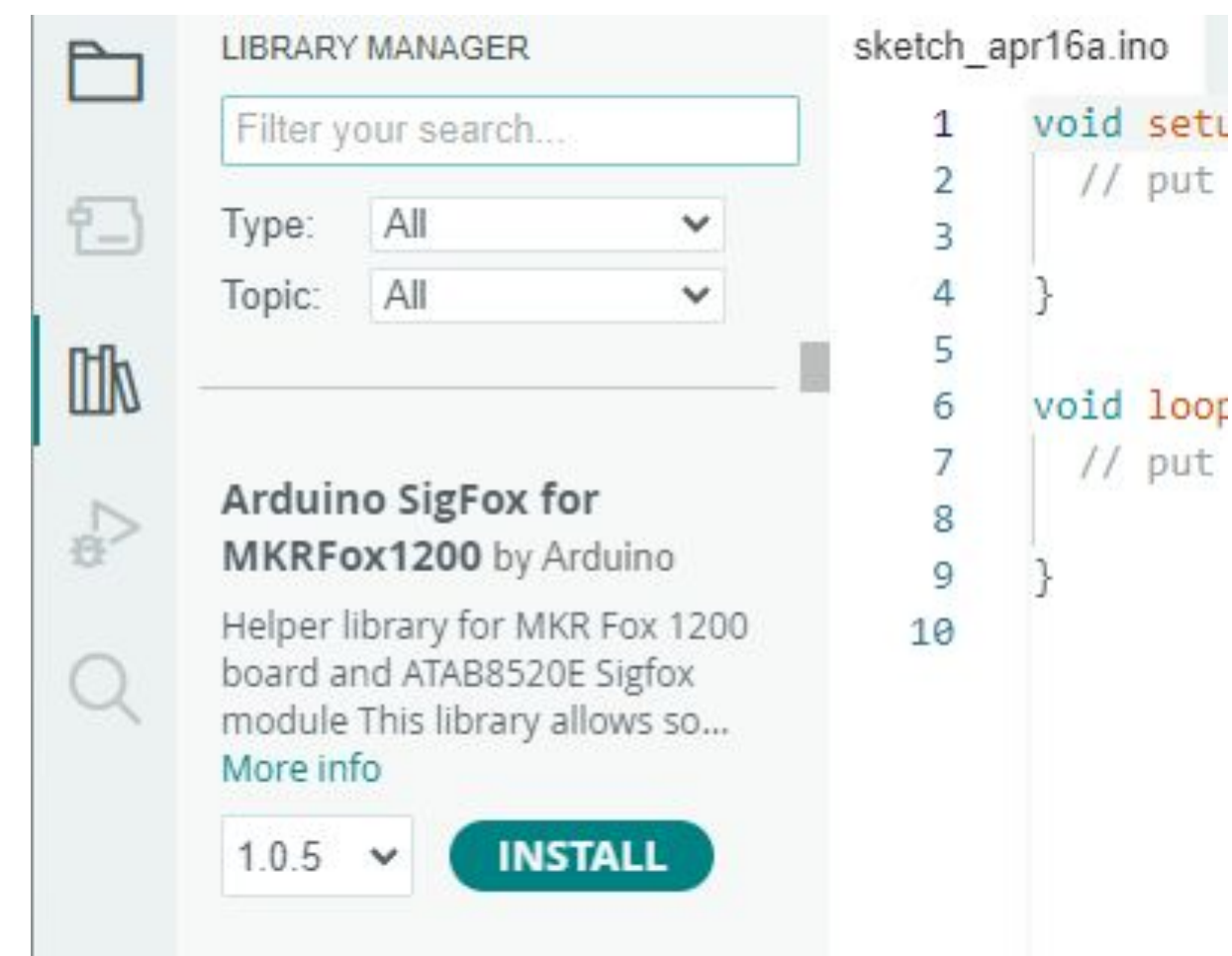


Conexión de puertos UART.

6. Librerías

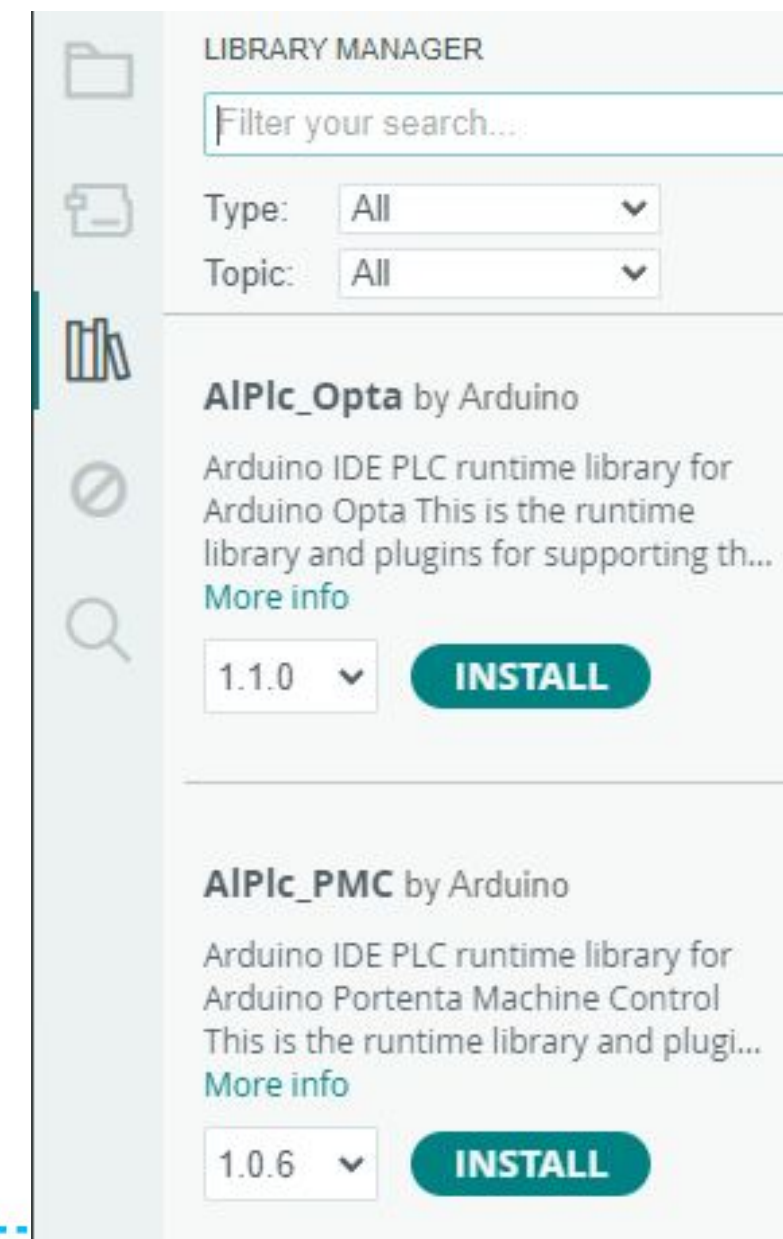
Las librerías son paquetes de código hechos por terceros que usamos en nuestro sketch. Esto nos facilita mucho la programación y hace que nuestro programa sea más sencillo de hacer y de entender.

- Communication (1458)
- Data Processing (418)
- Data Storage (177)
- Device Control (1191)
- Display (584)
- Other (597)
- Sensors (1364)
- Signal Input/Output (522)
- Timing (255)
- Uncategorized (253)



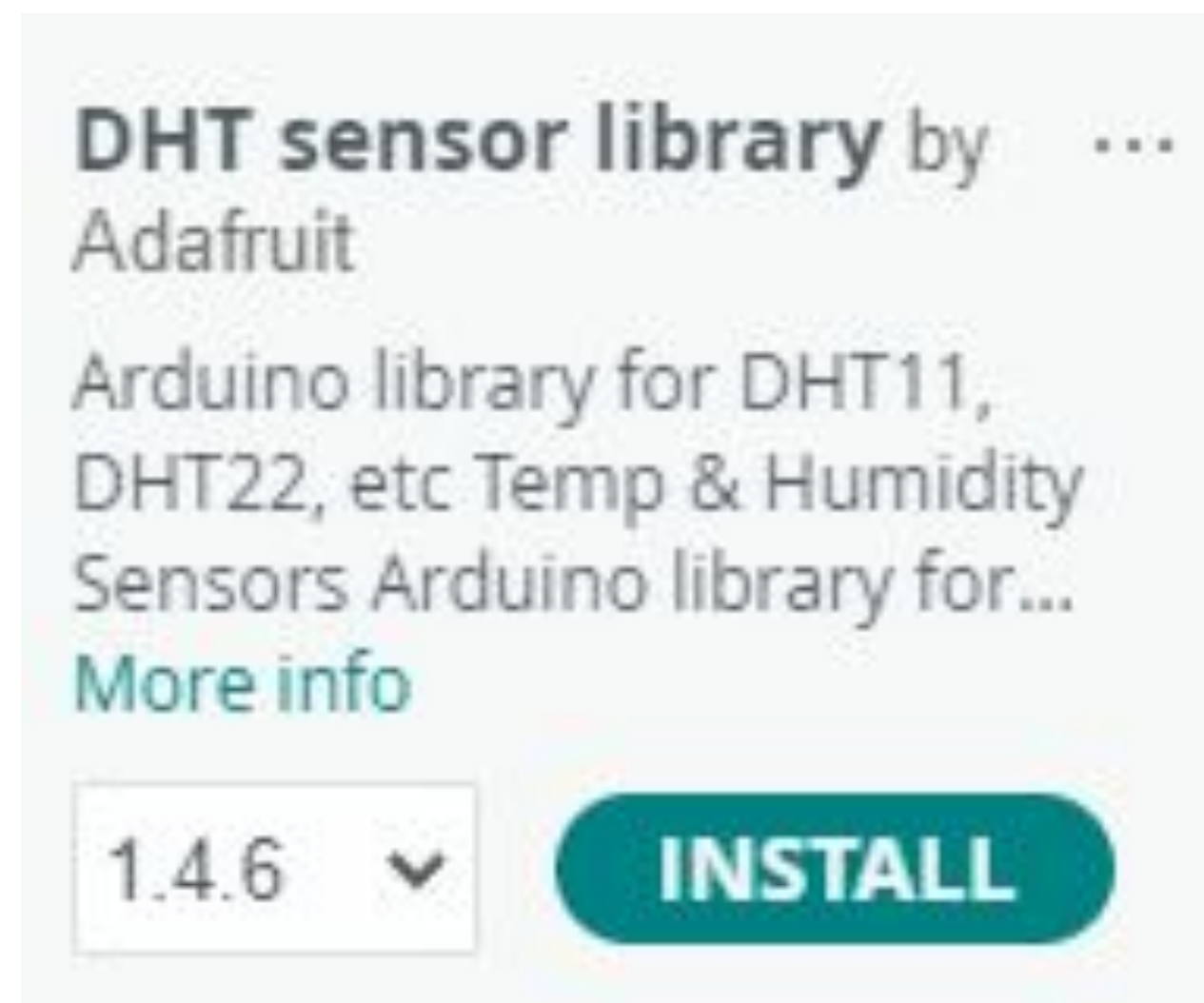
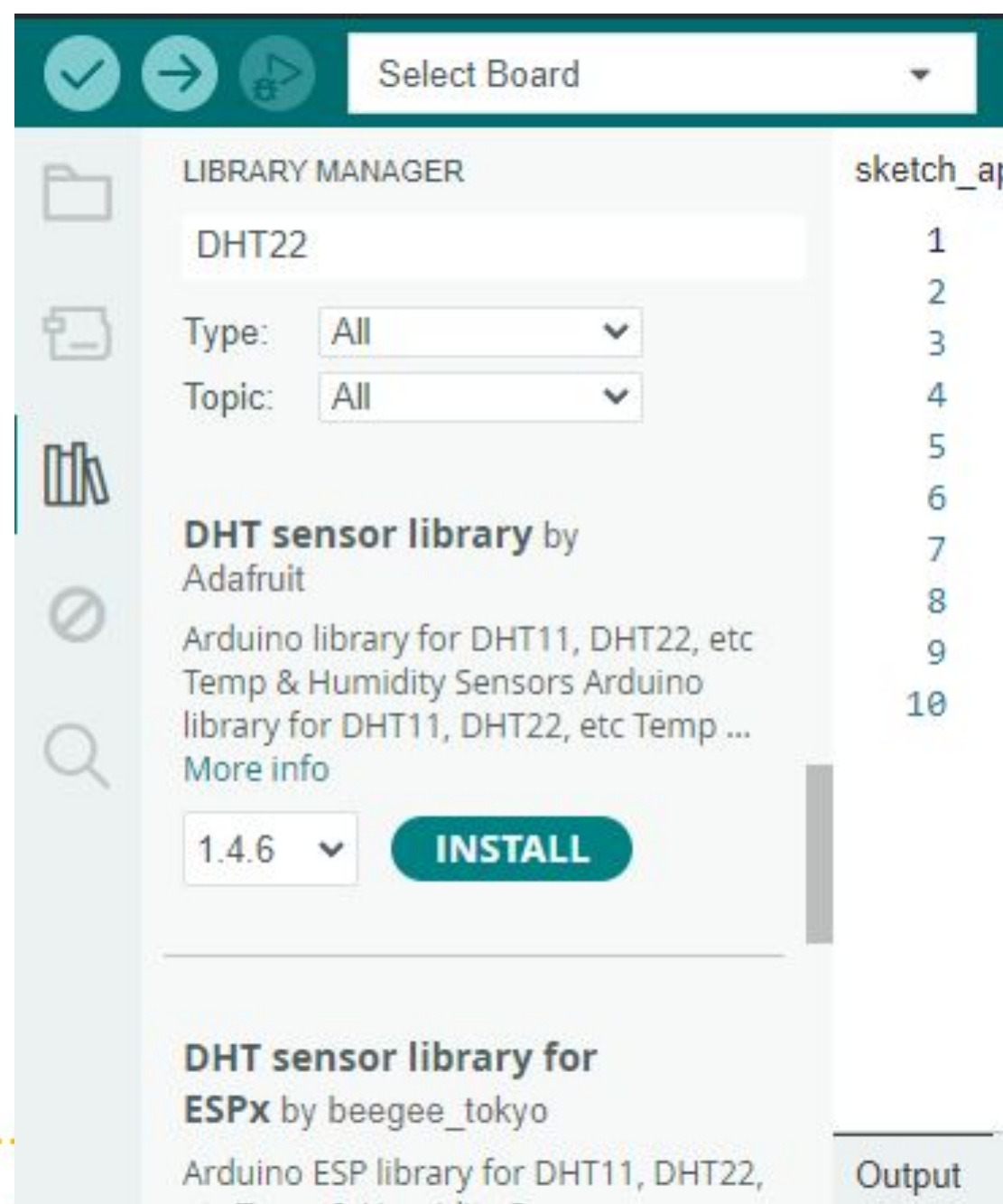
La instalación de librerías se puede hacer directamente desde el gestor de librerías o manualmente.

Así mismo ya existen librerías de diversos sensores en Arduino las cuales se pueden descargar en el **Administrador de Librerías**.



Por ejemplo para el sensor DHT22:

Buscamos el nombre del sensor en la barra de búsqueda:



Por ejemplo para el sensor DHT22:

Instalamos la librería, (instalar con todas las dependencias):

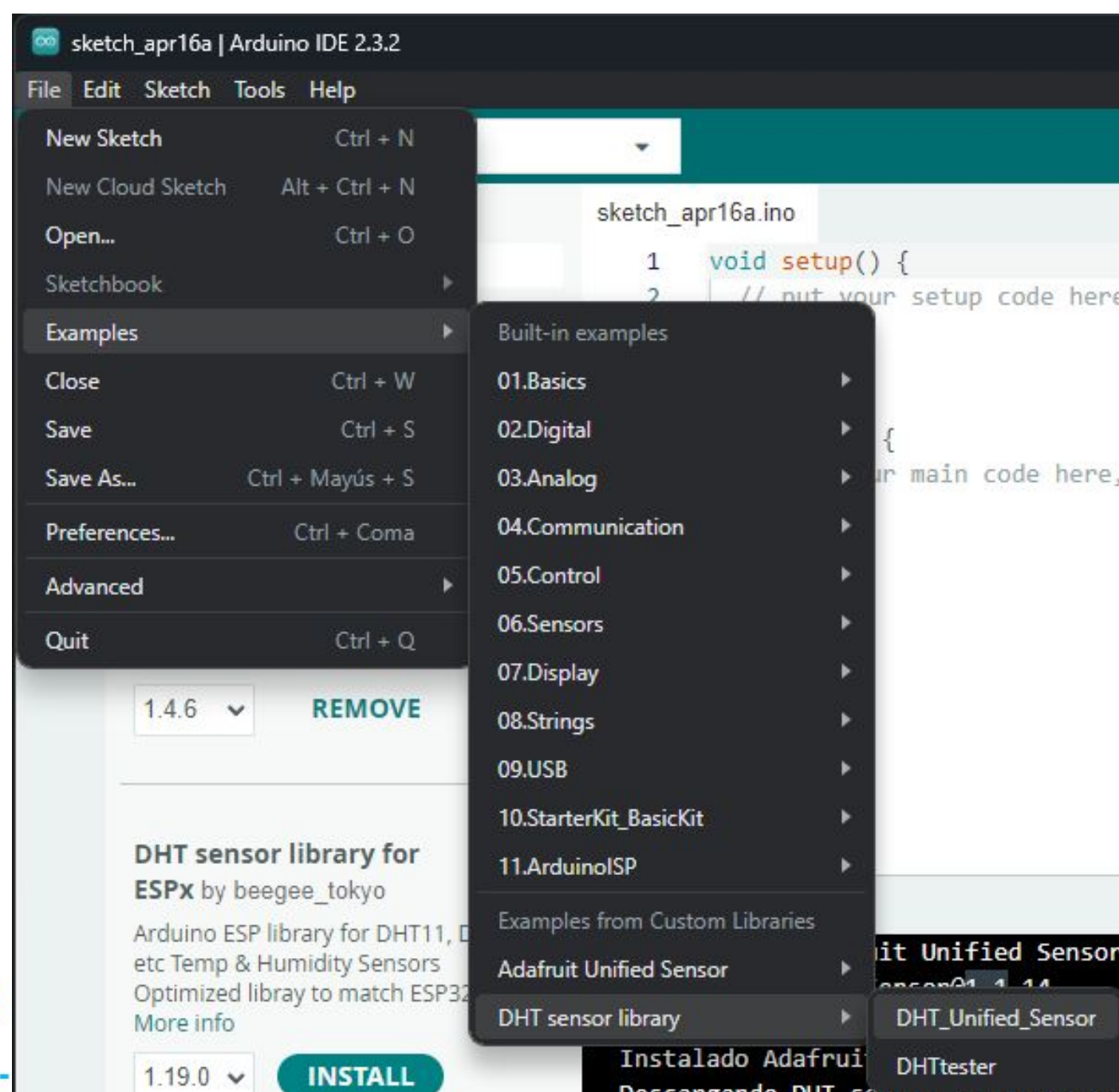


Output

```
Descargando Adafruit Unified Sensor@1.1.14
Adafruit Unified Sensor@1.1.14
Instalando Adafruit Unified Sensor@1.1.14
Instalado Adafruit Unified Sensor@1.1.14
Descargando DHT sensor library@1.4.6
DHT sensor library@1.4.6
Instalando DHT sensor library@1.4.6
Instalado DHT sensor library@1.4.6
```

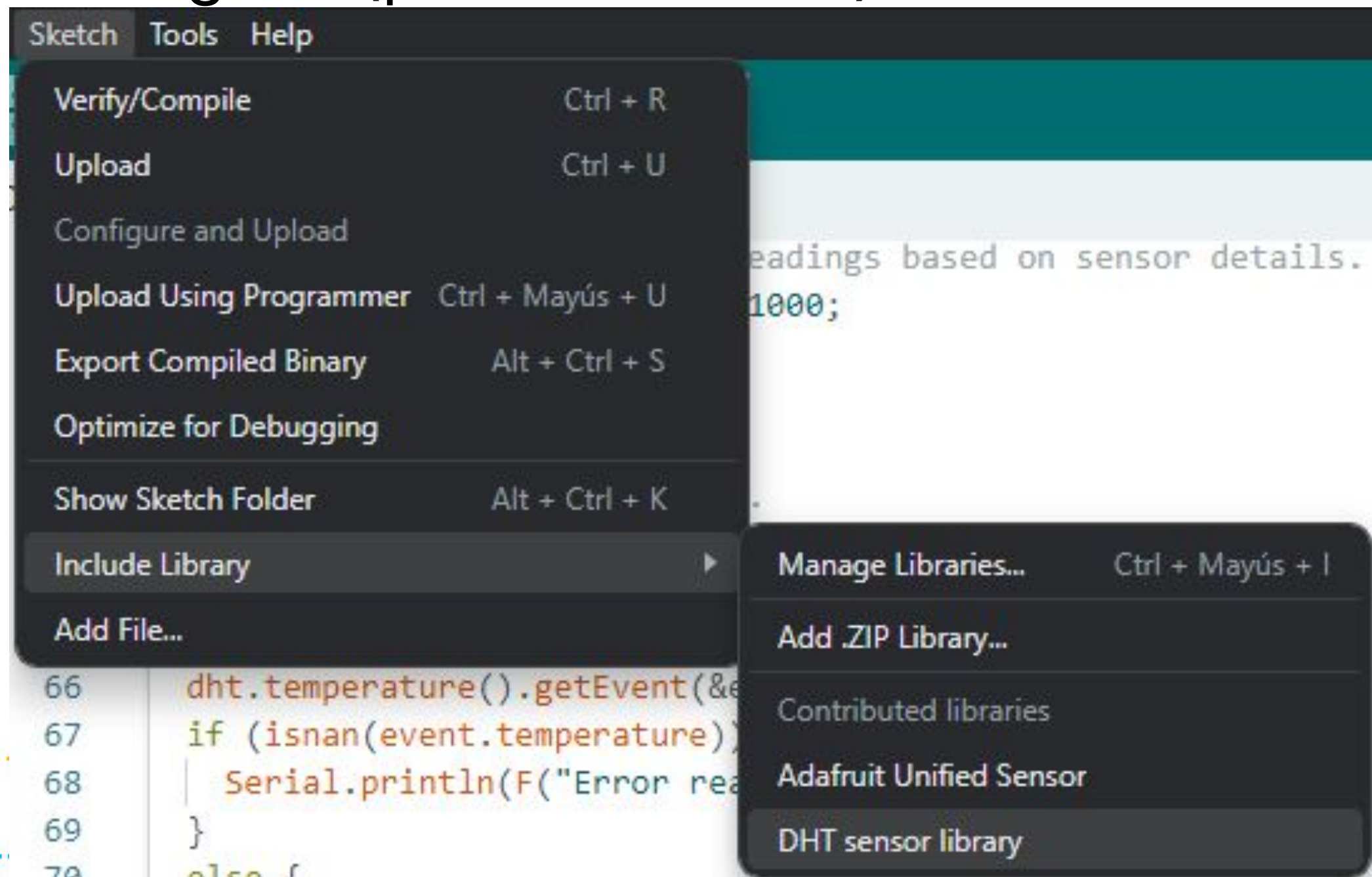

Por ejemplo para el sensor DHT22:

Luego podemos buscar un ejemplo para el sensor instalado (si la librería lo incluye)



Por ejemplo para el sensor DHT22:

También podemos buscar las librerías instaladas con el paquete descargado (pestaña Sketch).



```
#include <DHT.h>
```

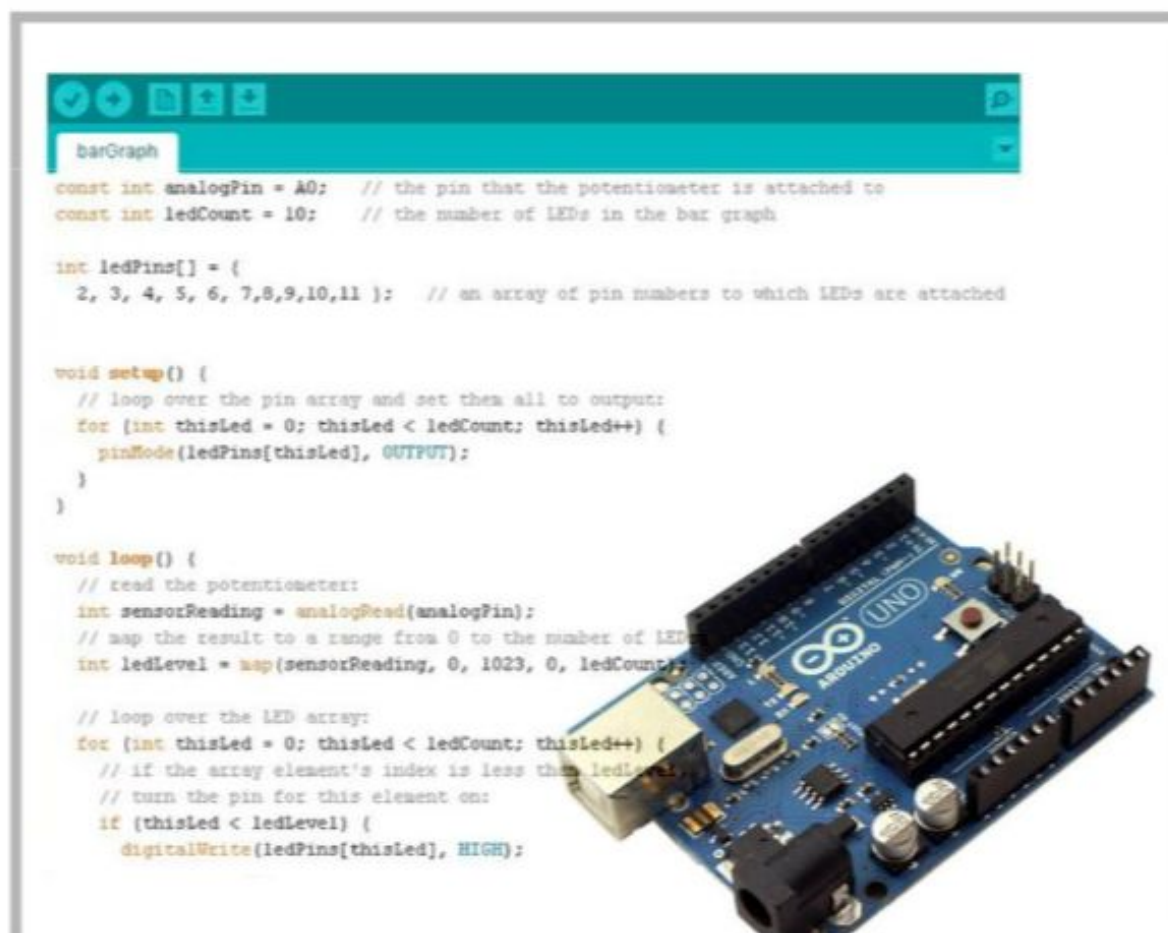
```
#include <DHT_U.h>
```


Guía básica de Arduino.

<https://www.calameo.com/read/001898612aa3ebf485e6e>

MANUAL DE ARDUINO

PROGRAMACIÓN Y CONCEPTOS BÁSICOS



2.



Comandos Básicos *Arduino (GPIO)*

Funciones de Pines Digitales

Los pines digitales operan exclusivamente con valores discretos: encendido (HIGH o 5V) y apagado (LOW o 0V). Para interactuar con ellos, usamos tres funciones fundamentales:

pinMode(pin, modo)

Es la configuración inicial. Le indica al microcontrolador cómo se comportará físicamente un pin. Siempre debe ir dentro del bloque setup().

- Parámetros: Recibe el número de pin y el modo, que puede ser OUTPUT (Salida: el Arduino entregará voltaje) o INPUT (Entrada: el Arduino detectará si hay voltaje externo).
- Ejemplo: **pinMode(3, OUTPUT);**
(Prepara el pin 3 para actuar como fuente de energía para un componente).



Funciones de Pines Digitales

Los pines digitales operan exclusivamente con valores discretos: encendido (HIGH o 5V) y apagado (LOW o 0V). Para interactuar con ellos, usamos tres funciones fundamentales:

digitalWrite(pin, estado)

Es la orden de acción para los pines configurados como salidas. Controla directamente el flujo de corriente.

- Parámetros: Recibe el número de pin y el estado deseado: HIGH (activa la salida a 5 voltios) o LOW (corta el voltaje a 0 voltios conectándolo a tierra).
- Ejemplo: **digitalWrite(3, HIGH);**
(Enciende el componente, como un LED, conectado al pin 3).



Funciones de Pines Digitales

Los pines digitales operan exclusivamente con valores discretos: encendido (HIGH o 5V) y apagado (LOW o 0V). Para interactuar con ellos, usamos tres funciones fundamentales:

digitalRead(pin)

Es la orden de lectura para los pines configurados como entradas. Funciona como un sensor que evalúa el voltaje presente.

- Retorno: Devuelve el valor lógico HIGH si detecta voltaje (usualmente más de 3V) o LOW si el voltaje es cercano a 0V.
- Ejemplo: `int estado = digitalRead(6);`
(Lee el pin 6 y guarda su estado en una variable para que el código pueda tomar decisiones, como saber si un botón fue presionado).



Monitor Serial

El Monitor Serial es una herramienta integrada en el entorno de desarrollo (IDE) de Arduino que permite la comunicación bidireccional entre la computadora y el microcontrolador a través del cable USB.

Sirve principalmente para ver en tiempo real qué valores están leyendo los sensores, en qué parte del código se encuentra la ejecución, o para que el usuario envíe comandos de texto hacia la placa.



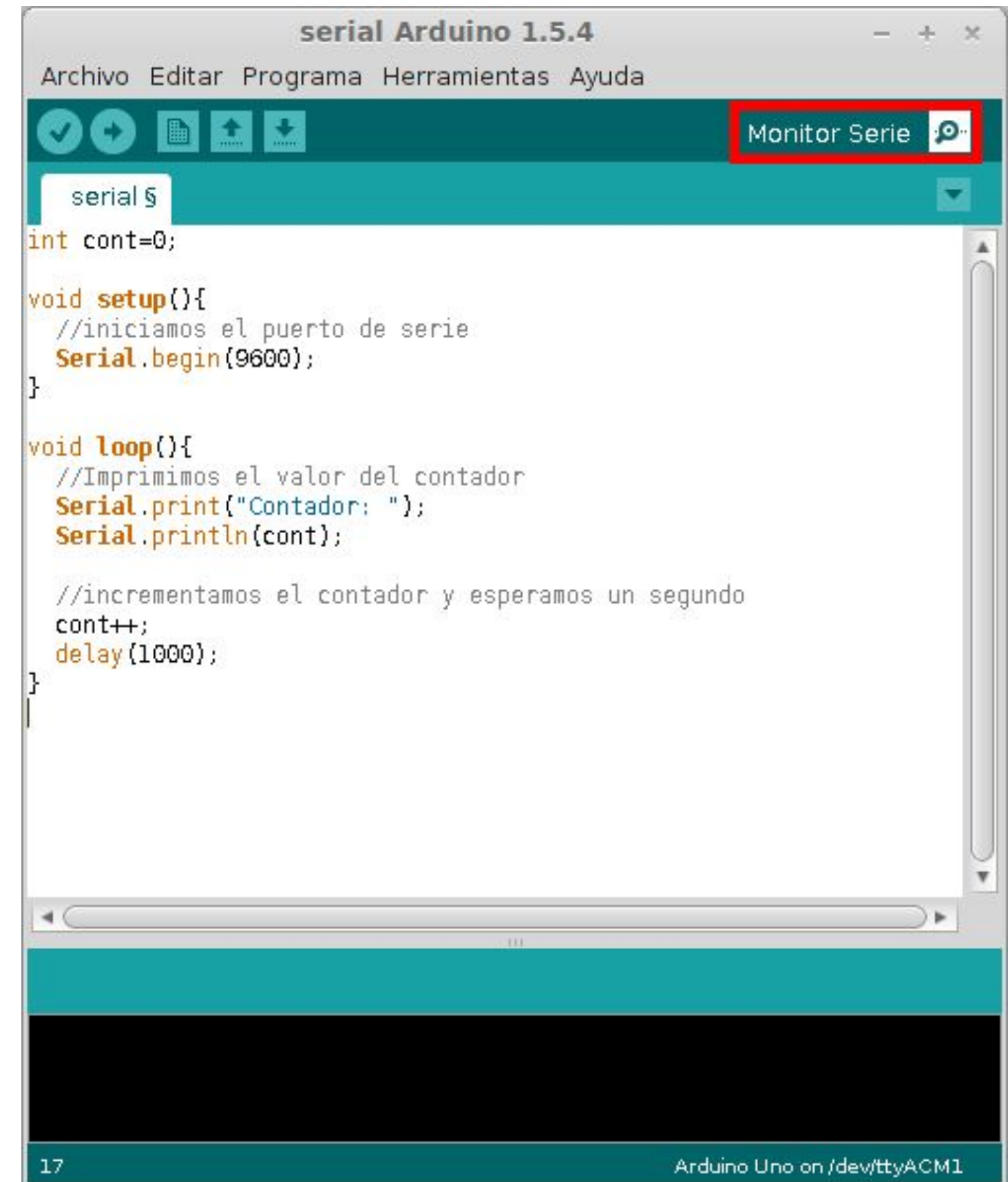
Nota: Cuando conectas el Arduino por USB a la computadora, un chip conversor en la placa realiza internamente esta conexión hacia los pines digitales 0 (RX) y 1 (TX).



Monitor Serial

El Monitor Serial es una herramienta integrada en el entorno de desarrollo (IDE) de Arduino que permite la comunicación bidireccional entre la computadora y el microcontrolador a través del cable USB.

Sirve principalmente para ver en tiempo real qué valores están leyendo los sensores, en qué parte del código se encuentra la ejecución, o para que el usuario envíe comandos de texto hacia la placa.



Monitor Serial

Para utilizarlo, primero se debe inicializar en el **setup()** con el comando:
Serial.begin(9600);

- **Baudrate:** El número entre paréntesis representa la tasa de baudios. Es la velocidad de comunicación medida en bits por segundo (bps).
- Tanto el código de Arduino como la ventana del Monitor Serial en la computadora deben estar ajustados a la misma velocidad.

Para enviar mensajes desde Arduino hacia la pantalla, se utilizan dos comandos durante el loop():

- **Serial.print("Hola");**
Imprime el texto de forma continua en la misma línea.
- **Serial.println("Hola");**
Imprime el texto y automáticamente da un salto de línea (como presionar "Enter").

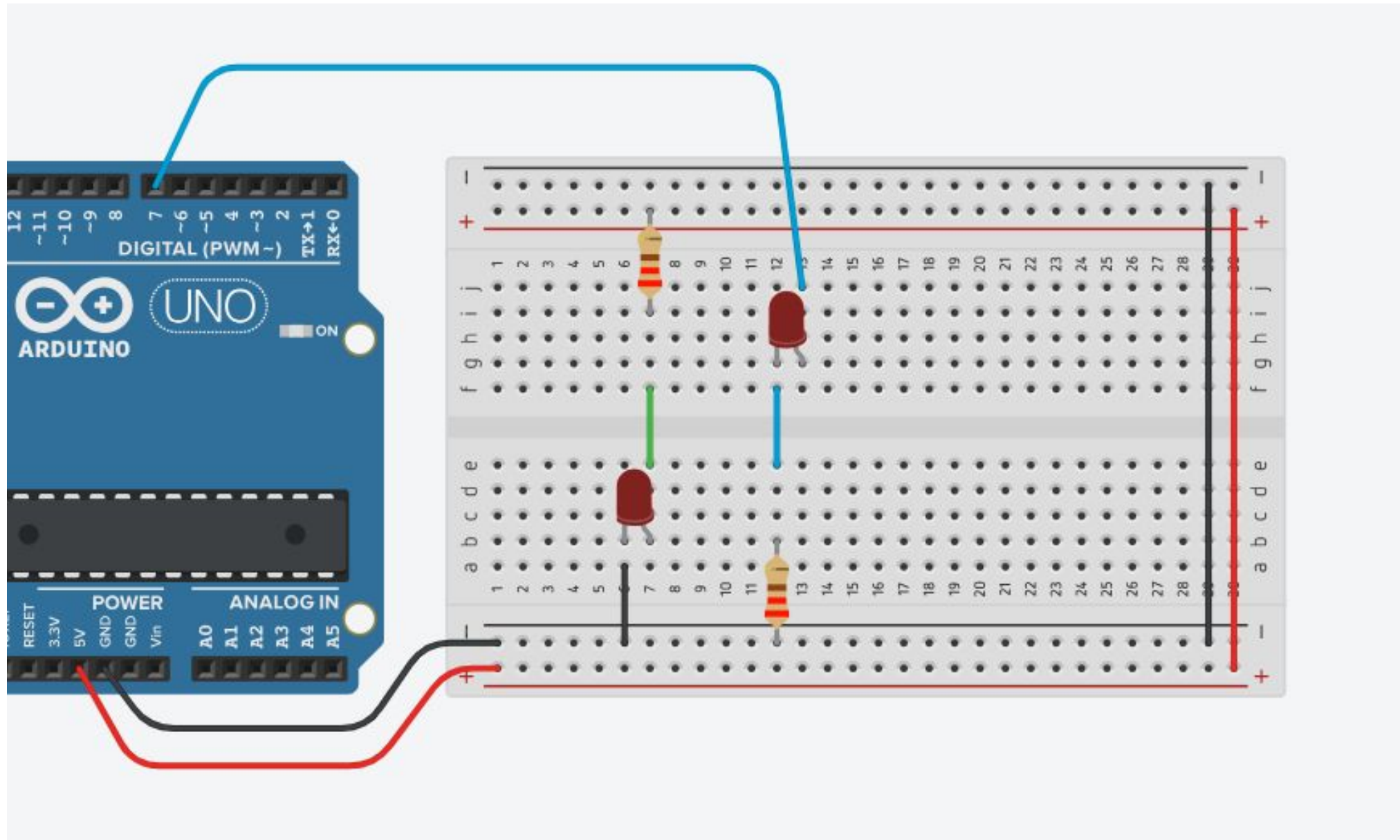


3.



Ejemplos de Programación *Arduino*

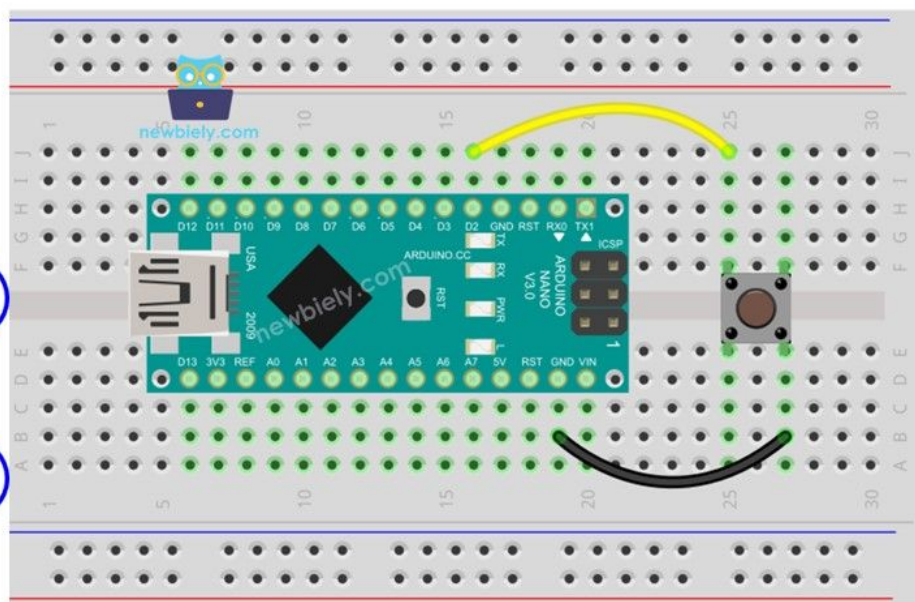
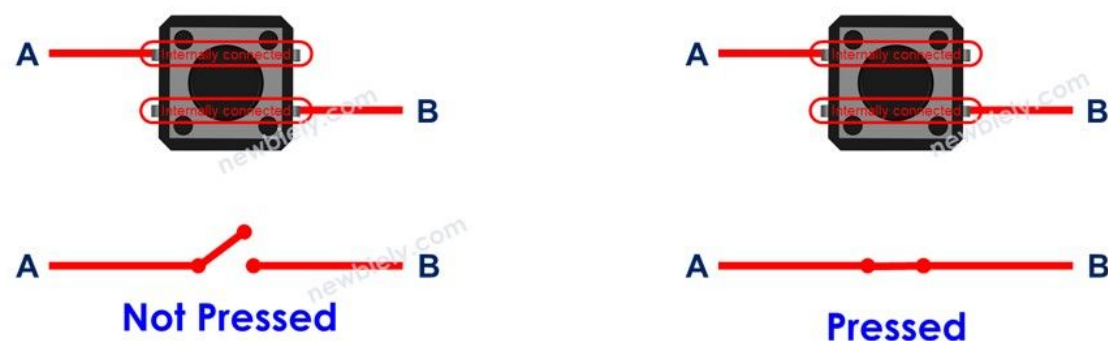
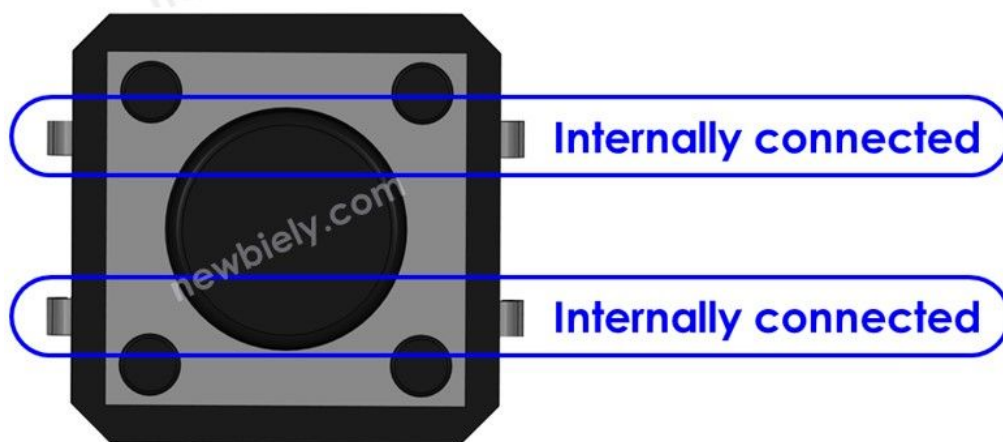
EJEMPLO I. Encendido Digital de LED



```
int LEDROJO = 3;
int LEDAZUL = 7;
void setup()
{
  pinMode(LEDAZUL, OUTPUT); // Define modo
                             digital (0 o 1).
}

void loop()
{
  int delayTime = 1000;
  digitalWrite(LEDAZUL, HIGH); // LOW OR HIGH |
                                0 o 1
  delay(delayTime);
  digitalWrite(LEDAZUL, LOW);
  delay(delayTime);
}
```

Botón con Arduino

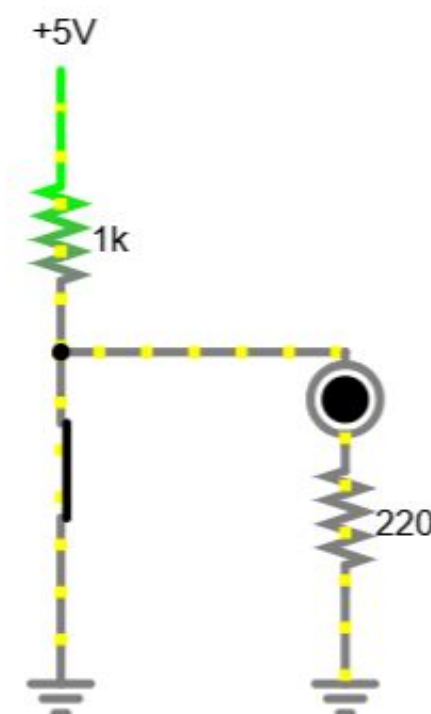
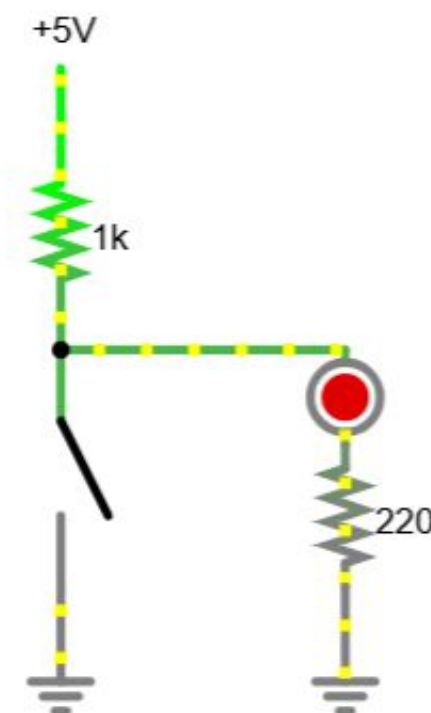


SIN PRESIONAR

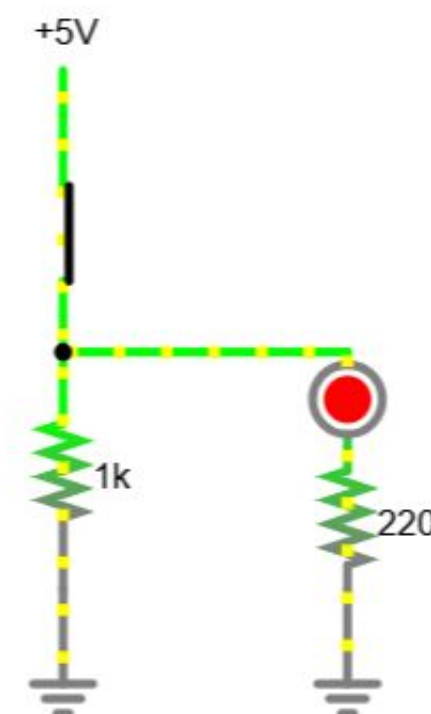
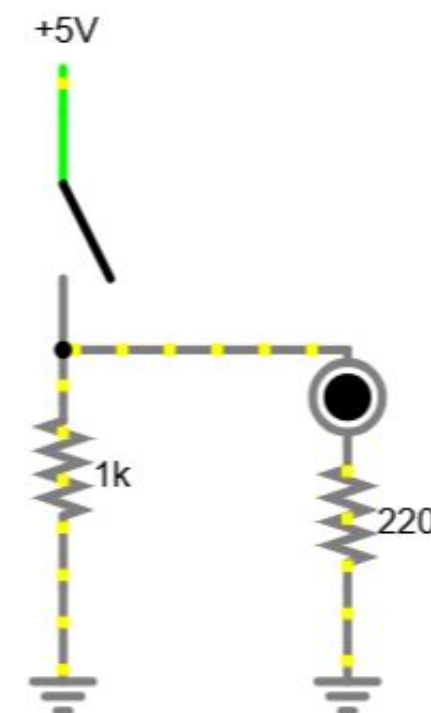
PRESIONANDO

> Reinventa el mundo <

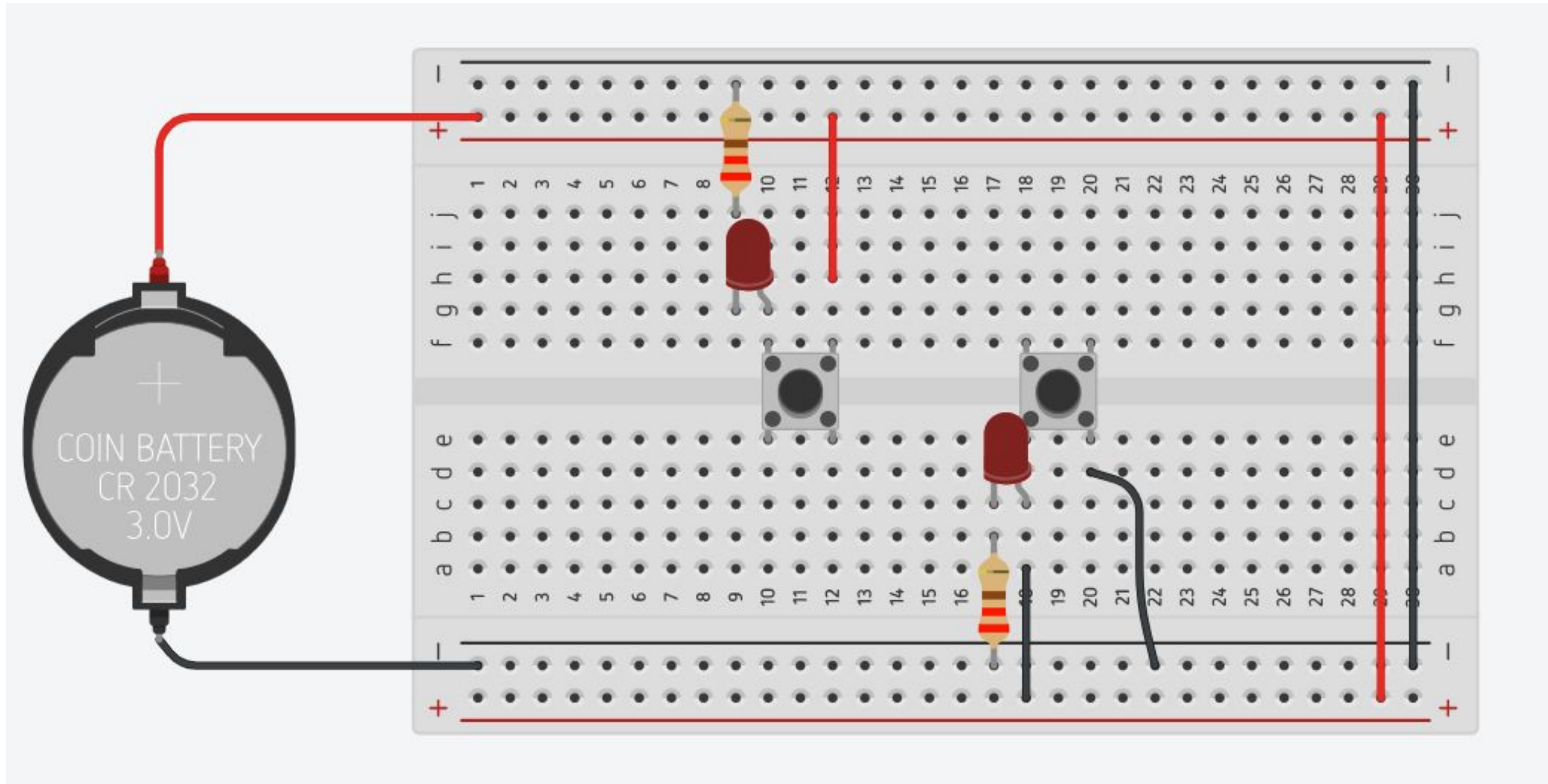
PULL-UP



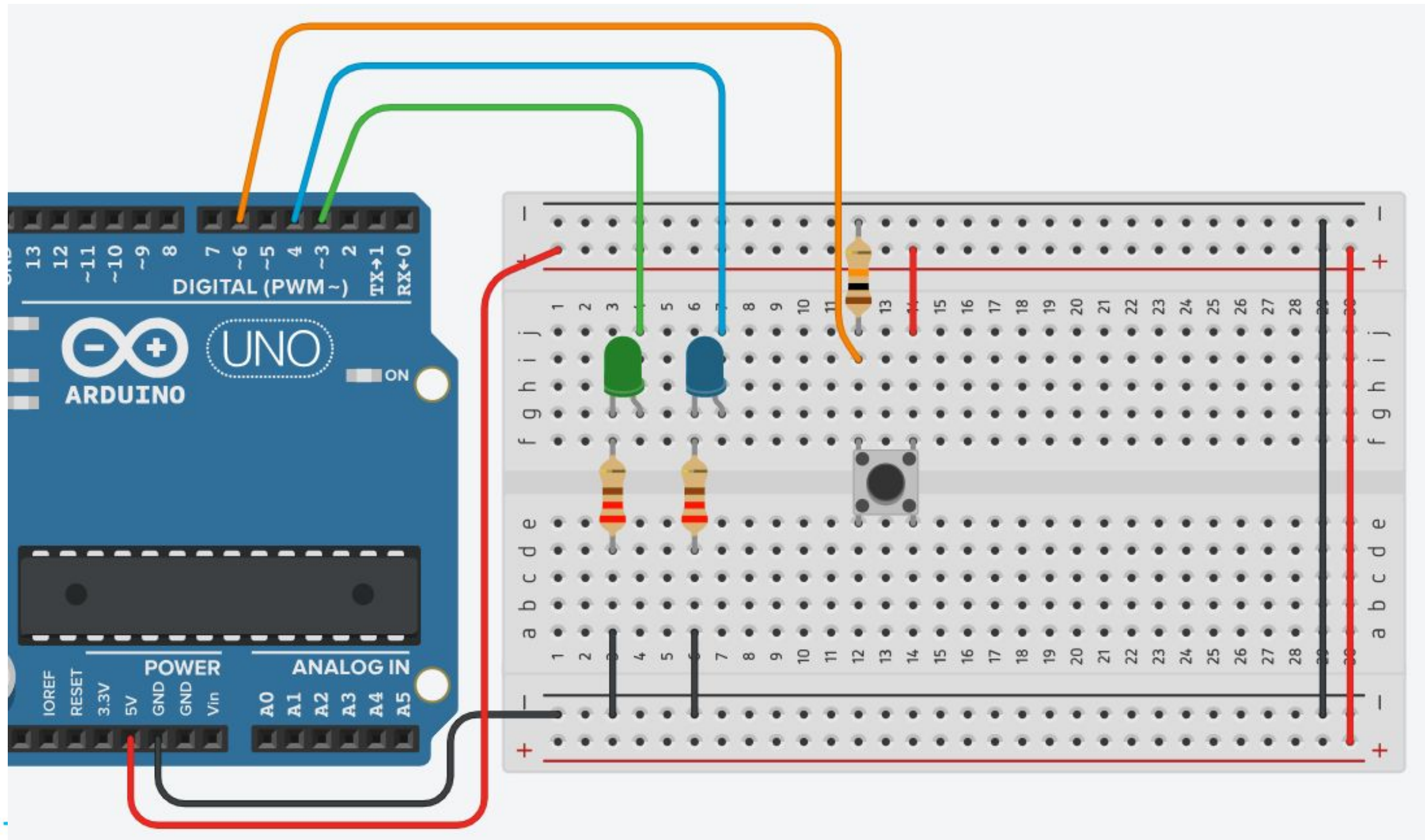
PULL-DOWN



Botón con Arduino



EJEMPLO II. Entradas y Salidas Digitales



EJEMPLO II. Entradas y Salidas Digitales

// 1. DEFINICIÓN DE PINES GLOBALES

```
// Usamos 'const int' (constante entera) porque los pines físicos no cambiarán
// mientras el programa se ejecuta. Esto protege la memoria y evita errores.
const int PIN_LED_VERDE = 3; // Actualizado para coincidir con el LED verde de la protoboard
const int PIN_LED_AZUL = 4;
const int PIN_BOTON = 6;
```

// 2. DECLARACIÓN DE FUNCIONES AUXILIARES

```
// Crear funciones propias ayuda a tener un código ordenado y no repetir líneas.
```

```
void parpadearLeds(int tiempoEncendido, int tiempoApagado) {
```

```
    // Encendemos ambos LEDs
    digitalWrite(PIN_LED_AZUL, HIGH);
    digitalWrite(PIN_LED_VERDE, HIGH);
    delay(tiempoEncendido);
```

```
    // Apagamos ambos LEDs
    digitalWrite(PIN_LED_AZUL, LOW);
    digitalWrite(PIN_LED_VERDE, LOW);
    delay(tiempoApagado);
}
```

// 3. CONFIGURACIÓN INICIAL (Se ejecuta solo una vez al encender el microcontrolador)

```
void setup() {
    // Configuración de los LEDs como SALIDA (OUTPUT)
    // El Arduino enviará voltaje (5V o 0V) a través de estos pines.
    pinMode(PIN_LED_VERDE, OUTPUT);
    pinMode(PIN_LED_AZUL, OUTPUT);
```

```
    // Configuración del botón como ENTRADA (INPUT)
    // El Arduino "leerá" el voltaje que ingresa por este pin.
    // Nota de hardware: El circuito usa una resistencia "pull-down", por lo que
    // leerá LOW (0V) en reposo, y HIGH (5V) al presionar.
    pinMode(PIN_BOTON, INPUT);
}
```

// 4. BUCLE PRINCIPAL (Iteración infinita)

```
void loop() {
    // Guardamos la lectura actual del botón en una variable local para evaluarla
    int estadoBoton = digitalRead(PIN_BOTON);
```

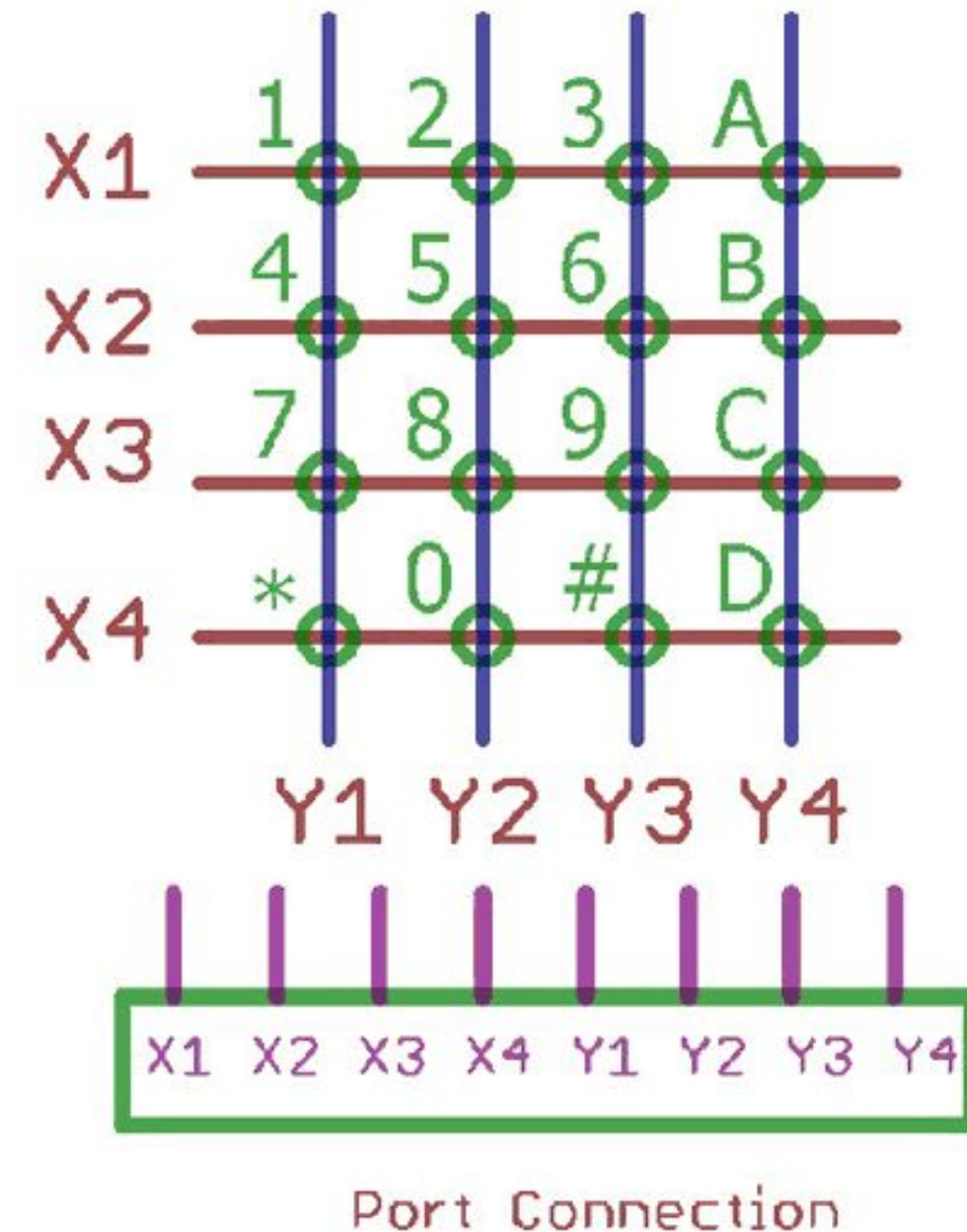
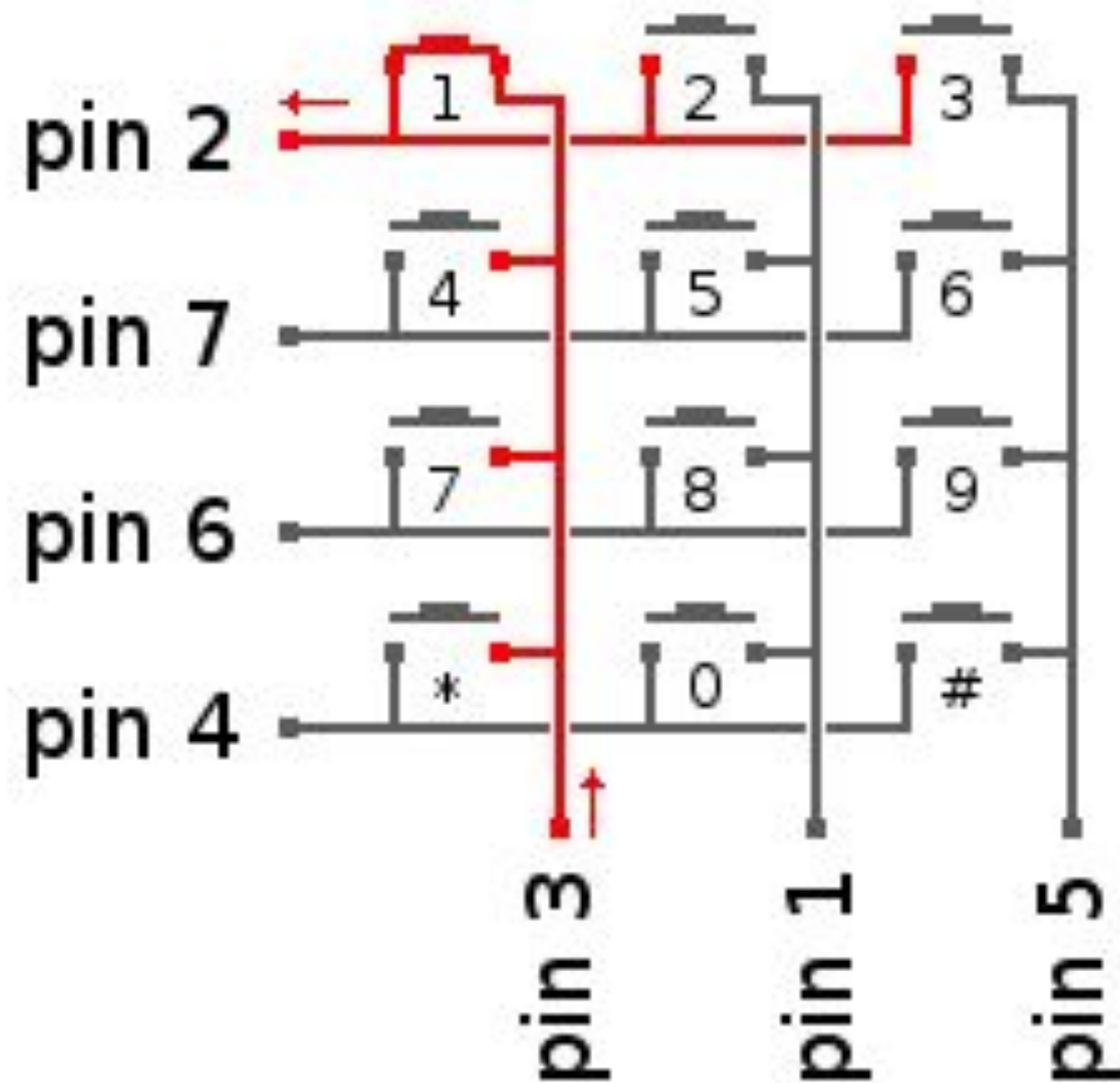
```
    if (estadoBoton == HIGH) {
        // CASO VERDADERO: El botón está presionado
        // Parpadeo asimétrico: 0.5s encendido, 3s apagado
        parpadearLeds(500, 3000);
    }
    else {
        // CASO FALSO: El botón NO está presionado
        // Parpadeo rápido simétrico: 0.1s encendido, 0.1s apagado
        parpadearLeds(100, 100);
    }
}
```



EJEMPLO III. Teclado Matricial con Arduino

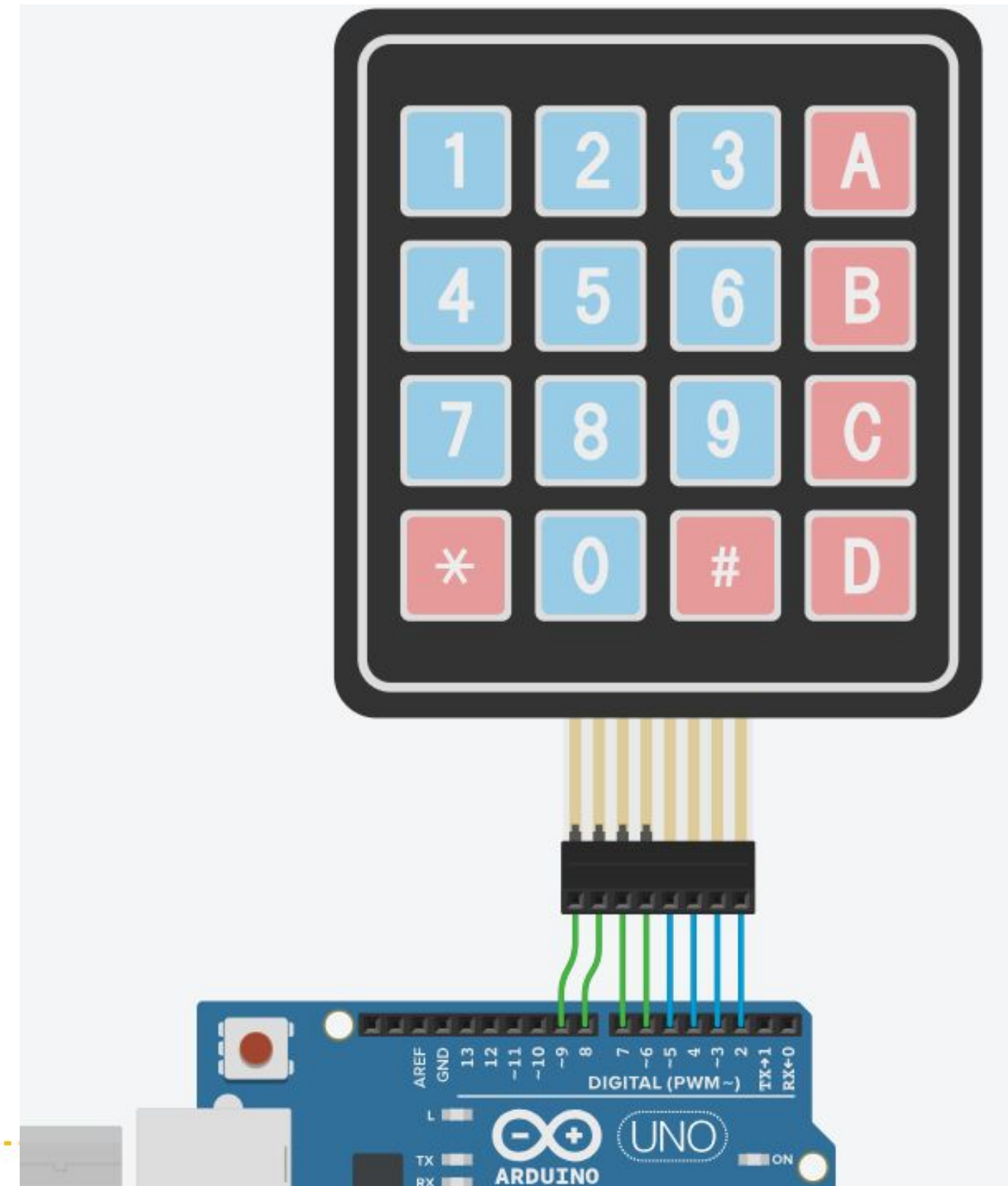
Usar referencia:

<https://controlautomaticoeducacion.com/sistemas-embebidos/arduino/teclado-matricial-keypad/>



EJEMPLO III. Teclado Matricial con Arduino

Una forma rápida de usar un Teclado con Arduino, es valernos de su librería Keypad (<https://www.arduino.cc/reference/en/libraries/keypad/>), bastante sencilla de entender, para el caso de un teclado matricial 4×4 (KEYPAD 4×4) tenemos



EJEMPLO III. Teclado Matricial con Arduino

Una forma rápida de usar un Teclado con Arduino, es valernos de su librería Keypad (<https://www.arduino.cc/reference/en/libraries/keypad/>), bastante sencilla de entender, para el caso de un teclado matricial 4x4 (KEYPAD 4x4) tenemos

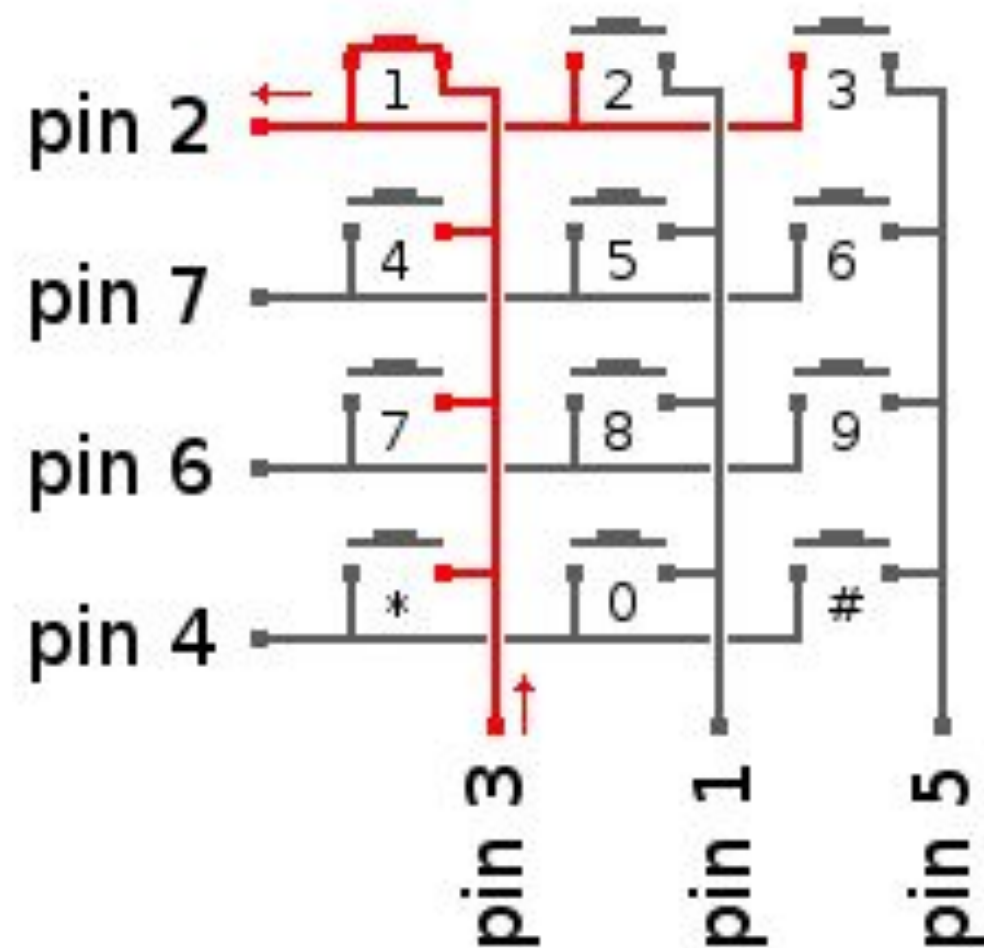
```
#include <Keypad.h>
const byte filas = 4;
const byte columnas = 4;
byte pinesFilas[] = {9,8,7,6};
byte pinesColumnas[] = {5,4,3,2};
char teclas[4][4] = {{ '1','2','3','A'},
                     { '4','5','6','B'},
                     { '7','8','9','C'},
                     { '*', '0','#','D'} };

Keypad teclado1 = Keypad( makeKeymap(teclas), pinesFilas, pinesColumnas,
                          filas, columnas);
void setup() {
  Serial.begin(9600);
  Serial.println("Teclado 4x4 con Biblioteca Keypad");
  Serial.println();
}
void loop() {
  //Verifica si alguna tecla fue presionada
  char tecla_presionada = teclado1.getKey();

  //Monitor Serial
  if (tecla_presionada)
  {
    Serial.print("Tecla: ");
    Serial.println(tecla_presionada);
  }
}
```



Uso de teclado matricial con Arduino.



```

#include <Keypad.h>
const byte filas = 4;
const byte columnas = 4;
byte pinesFilas[] = {9,8,7,6};
byte pinesColumnas[] = {5,4,3,2};
char teclas[4][4] = {{ '1','2','3','A'},
                     { '4','5','6','B'},
                     { '7','8','9','C'},
                     { '*', '0', '#', 'D' } };

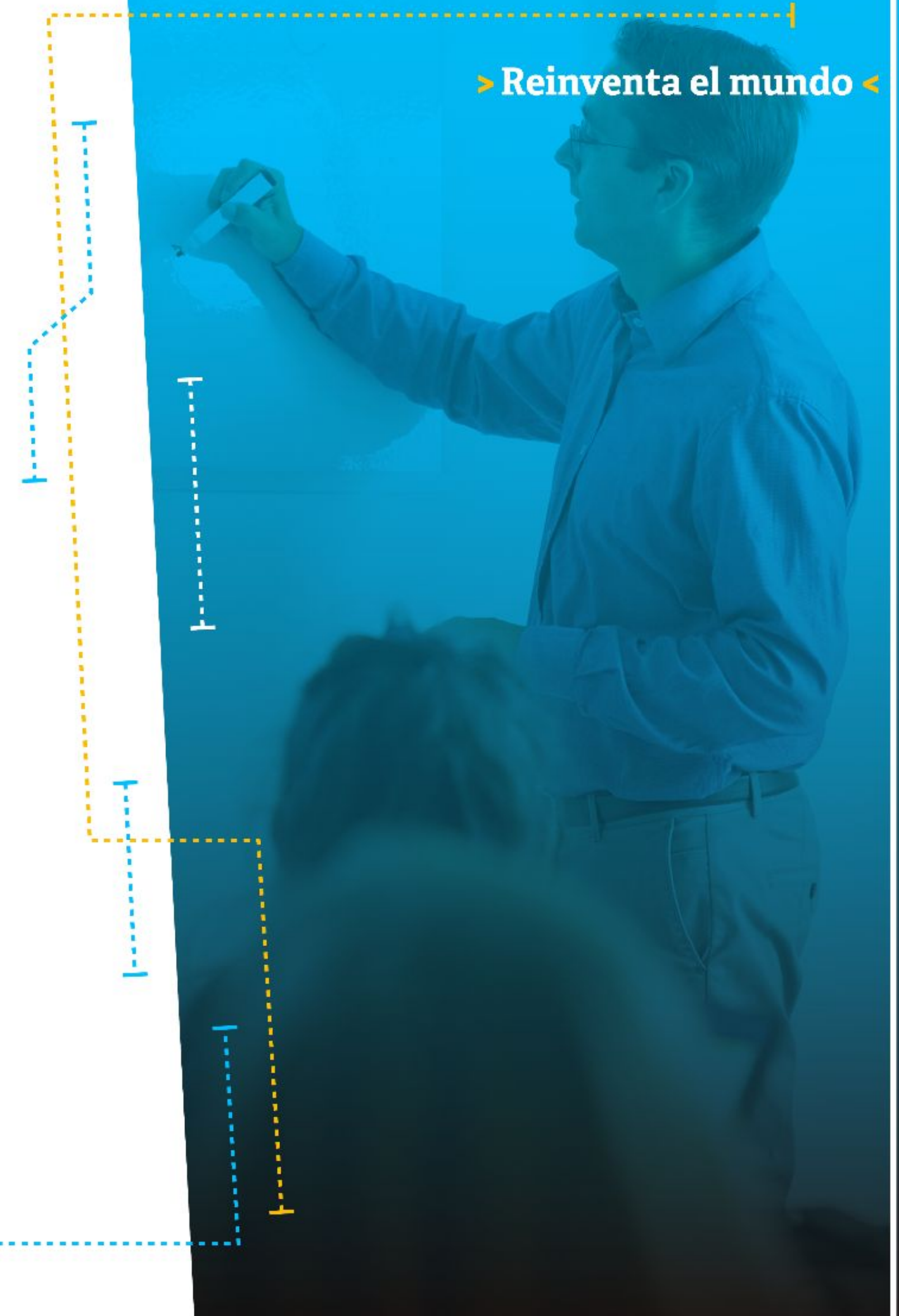
Keypad teclado1 = Keypad( makeKeymap(teclas), pinesFilas,
pinesColumnas, filas, columnas);
void setup() {
  Serial.begin(9600);
  Serial.println("Teclado 4x4 con Biblioteca Keypad");
  Serial.println();
}
void loop() {
  //Verifica si alguna tecla fue presionada
  char tecla_presionada = teclado1.getKey();
  //Monitor Serial
  if (tecla_presionada)
  {
    Serial.print("Tecla: ");
    Serial.println(tecla_presionada);
  }
}

```


Conclusiones

- ➔ **Aplicación de las Leyes de Kirchhoff.** Se ha reforzado el uso de las Leyes de Kirchhoff para el análisis de circuitos eléctricos, permitiendo determinar corrientes y tensiones en distintos nodos y mallas.
- ➔ **Cálculo de Resistencia Equivalente.** Se han aplicado métodos de reducción de resistencias en serie y paralelo para determinar la resistencia equivalente en circuitos con múltiples componentes.
- ➔ **Análisis de Circuitos con Diodos.** Se ha estudiado el comportamiento de diodos de silicio y germanio en circuitos eléctricos, identificando las condiciones de conducción y determinando las corrientes y potencias asociadas.
- ➔ **Evaluación de Potencia y Consumo Energético.** Se ha calculado la potencia consumida por los distintos elementos del circuito y la potencia suministrada por la fuente, lo que permite analizar la eficiencia del sistema.

Ref. Boylestad (2004) Introducción al análisis de circuitos Eléctricos.



GRACIAS

Internet de las Cosas 2025-I

